

EEL4783: Final Project Part 2

Project Title: "The 4-Element Dot Product Engine"

Project Objective

Demonstrate mastery of HDL behavioral modeling, testbench verification, and design trade-offs by implementing a **4-Element Dot Product Accelerator**. You may choose to implement this in **RTL (VHDL/Verilog/SystemVerilog)** or **High-Level Synthesis (SystemC/C++)**.

This project ties together course topics:

- **Behavioral Modeling** (Lectures 3–6)
- **Arithmetic Datapaths** (CLA/Multipliers from Midterm)
- **SystemC/HLS** (Lectures 8–12)
- **Verification** (Testbenches)

Problem Statement

Design a digital module that computes the dot product of two 4-element vectors: $Y = (A_0 \times B_0) + (A_1 \times B_1) + (A_2 \times B_2) + (A_3 \times B_3)$

Specifications

Parameter	Requirement
Input Data	Two vectors: A[3:0] and B[3:0]
Data Width	8-bit signed integers (2's complement)
Output Data	16-bit signed integer (to prevent overflow)
Clock	Single positive-edge triggered clock (clk)
Reset	Active-high synchronous reset (rst)
Control	Single-cycle computation OR Multi-cycle pipeline (your choice)
Language	Choose ONE: VHDL, Verilog, SystemVerilog, or SystemC (HLS)

Implementation Tracks

♦ Task 1: RTL Design (VHDL / Verilog / SystemVerilog)

- **Focus:** Structural/Behavioral modeling, explicit state control.
- **Requirement:** Must include a testbench that verifies all corner cases (negative numbers, zero, max values).
- **Challenge:** Implement a **pipelined** version (multiply stage → add stage) for extra credit.

♦ Task 2: HLS Design (SystemC / C++)

- **Focus:** Algorithmic description, synthesis directives.
- **Requirement:** Use HLS tool (e.g., Vitis HLS, Catapult, or Simulink HDL Coder) to generate RTL.
- **Challenge:** Use pragmas/directives (#pragma HLS PIPELINE, UNROLL) to optimize for **Latency** vs. **Area** and report the difference.

Deliverables (Due in 1 Week)

Item	Format	Details
1. Source Code	.v, .vhd, .cpp, .sv	Well-commented, parameterized where possible
2. Testbench	Same language	Must verify ≥ 10 unique test cases (including negatives)
3. Simulation Waveforms	.png or .pdf	Screenshot showing inputs, outputs, and clock for 3 test cases
4. One-Page Report	.pdf	See template below
5. README	.txt	Instructions to compile/run your design

Report Template (1 Page Max)

1. **Design Choice:** Why did you choose RTL vs. HLS? (2 pts)
 2. **Architecture:** Block diagram or flowchart of your implementation. (3 pts)
 3. **Verification:** How did you verify correctness? (2 pts)
 4. **Reflection:** What was the hardest part? What would you improve? (3 pts)
-

Grading Rubric (100 Points)

Category	Points	Criteria
Functionality	50 pts	Code compiles/synthesizes; Output matches expected math for all test cases
Testbench	20 pts	Comprehensive coverage (negatives, zero, overflow checks); Self-checking preferred
Code Quality	15 pts	Clear naming, comments, proper indentation, parameterization
Report	10 pts	Clear explanations, diagram, reflection on trade-offs
Submission	5 pts	On time, correct file structure, README included

Extra Credit (Up to +10 pts)

- **Pipelining (RTL):** Implement a 2-stage pipeline and show improved F_{max} in synthesis report.
- **HLS Optimization (HLS):** Show two synthesis results (Area-optimized vs. Speed-optimized) and compare resource usage.
- **SystemVerilog Assertions:** Use SVA (assert property) in your testbench for automatic checking.